

Emanuel Sanchez

CS3300 - 22205

Programming Project - Research Paper

My project is a gallery of games that the users can log into to play. The games consist of two-player and single-player games. The single-player games create scores that are kept track of in files to show the current record holder for that game; this is also why the user's account is password-protected.

I chose this project because we were given a couple of months to work on a Java project. However, I knew that working on something for that long requires the program to be extremely big. This instead led me to start working on something modular. After working on the general program, such as the users and high score manager, I worked on games and kept adding as many games as I had time for until the project was due. Overall, this project was a really fun experience, and I was able to learn a lot about how Java programs work. I have always preferred coding using C++, but not having to think too much about memory allocation because of the garbage collector in Java has made this a unique experience for me. I also had to get used to the idea that almost all variables were objects already. This felt weird to me at first, but I was able to use this to my advantage in a lot of places while coding my program. All in all, I definitely enjoyed coding in Java and saw a lot of its similarities with C++ and it broadened my scope on what I can do with different coding languages. For example, I have always known the main advantage of Java was its portability, but never looked into it as much as I should. I have now successfully transplanted this game into one of my other projects that I have outside of school to be able to take around and show friends.

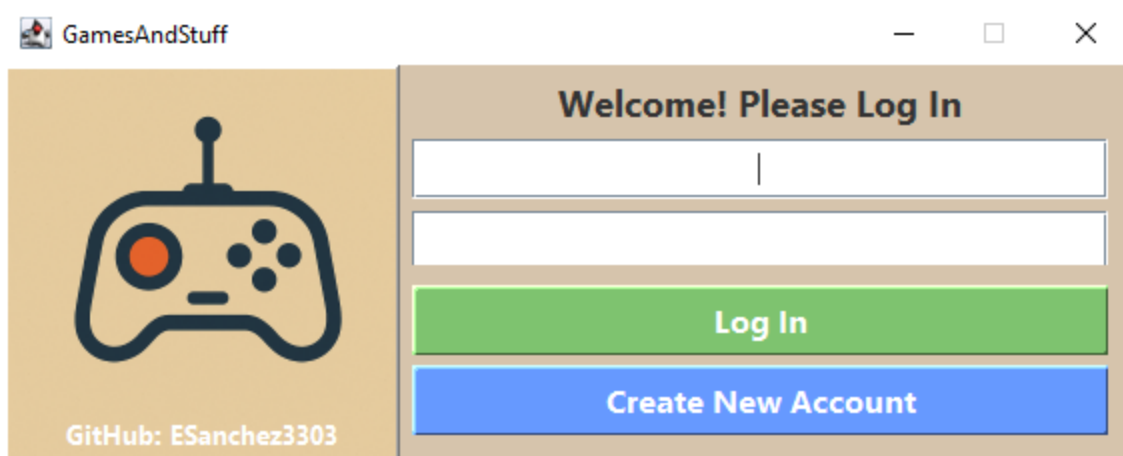
The program does not compute anything specific besides local variables that are used for visuals of the games. The inputs for the program are mostly mouse clicks from a user and game inputs that are used to play the game. Below is a short version of how each part of the program works, including the main GUI window, each game, and the managers behind the scenes who keep track of the high scores and users.

We will start with the main GUI window, called **GUI_window.java**:

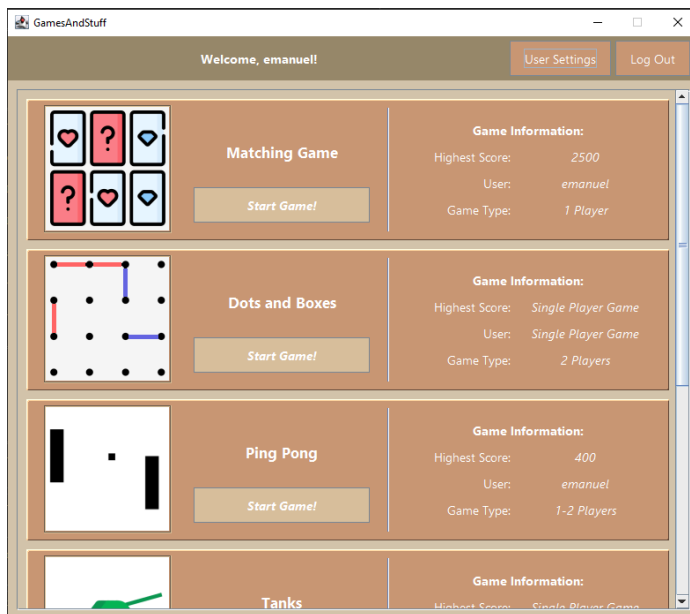
Note: The initComponents() in the beginning and variable declarations at the end are generated automatically by NetBeans when using the Form Designer.

- 1) GUI_window() : constructor
 - a) This constructor is called in another file called “startingClass.java”, which makes the object of the entire form. It is a regular constructor call that calls the initComponents() and then the rest of the code that I made on there
 - b) The first thing it does is add an action listener to the password input field in the login screen. This is because I forgot to add a function when I was making the login section, and instead of going back or making the call for this function somewhere else, making the code look unorganized, I just added the listener in the constructor. This listener calls for the function that I could place next to the login section of the code to keep everything organized.
 - c) The next section calls the “Constructor Functions” of each game. Since we have to wait for the variables to be initialized in the initComponents() function, we had to make a function after the construction of each game object. The function is called setUp() in each game, and I call it to send in the objects that the game needs from the main GUI window.
 - d) Each game has its own unique needs from the GUI window, so those are sent in
 - e) Aside from this, the constructor also does some one-time code that is meant to fix problems in the window, such as setting a spinner to be a “double” instead of an “int” because I was unable to do so in the designer.
- 2) This class also has an instance of the UserManager, the HighscoreManager, and each game.
- 3) The other main function that we use here is the switchFrame(JPanel) function. This is a very important function because it is needed to provide the transition from one state of the program to the next. The function takes in a JPanel to which we want to show next. The function starts by hiding all other panels, then sets up the target panel as we want it to look to the user, and then shows that panel. This way, we can organize the program in a way that lets us move from panel to panel while resetting the panel back to its original starting state.
- 4) Every other function in this file responds to interactions with the form. For example, if the user clicks the login button, then the program grabs the username text and the password text, and sends them to the UserManager that checks if this combination is valid.

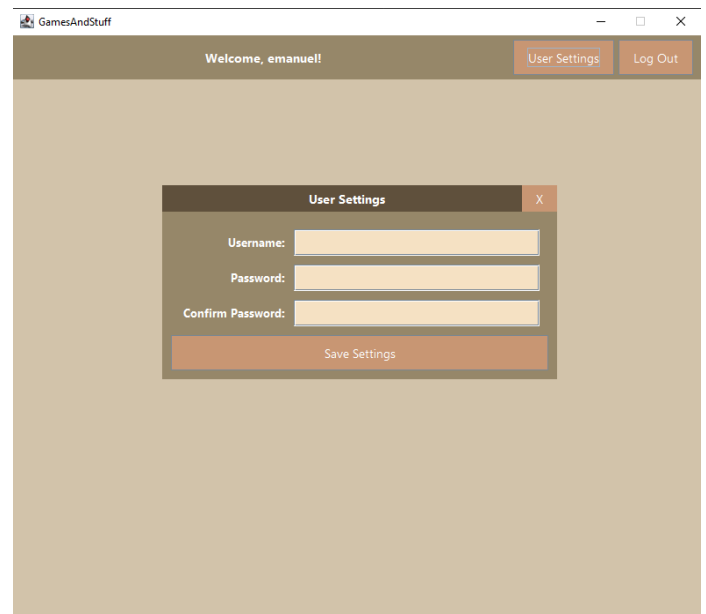
Going through each of these will make this file description very long. However, we will go through each game, which will touch briefly on each function in this main GUI file.



This is the GUI log in window that shows up when the program first starts.



This is the GUI that the user would see after they login. The user can click any game from here and also can see the high score that is currently set by them or another user.



This is the user settings that lets the user change their username or password. This section is coded to detect which one the user is trying to change. Once it is changed, the user is made aware and saves it all using the User Manager.

Next up, we will go through the **UsersManager.java** file:

- 1) This file has two classes: User and UsersManager. The User class is made to hold the information of one user, which will then be the type for a list that is a variable in UserManager.
- 2) Users only hold, set, and get their own username and password.
- 3) When an instance of UserManager is constructed in the *GUI_window.java* file, it resets the user List of type <User>. Then it calls the loadFromFile() function.
- 4) The loadFromFile() function's purpose is to read the file "users.txt" to get the current users that we have at the moment.
- 5) However, we do encrypt the passwords before saving them in the file. Therefore, we need to decrypt them as we read them into the program. The encryption is a simple shift of the alphabet by the amount saved in: "shift".
- 6) In this function, we also make a file if no file is found.
- 7) We read the line grabbed from the file, which is in the format of:
<username>,<encryptedPassword>
- 8) It does this for as many times as there are lines, which is also how many users have been created.
- 9) After the List<User> is made, it stays there until it needs to be read as requested. This is done when the user in the form types in their credentials, and then the program calls checkCredentials(). This function checks the list that we have and checks if the combination is valid.
- 10) As far as making a new user, we need to check if the username is valid so that each user can have a unique username. We have the function of validUsername() that does this and returns true/false accordingly.
- 11) Besides some other helper functions, we lastly have saveToFile(), which is called every time after a change is made to the list. Ex: addUser(), setUsername(), setPassword().
- 12) The saveToFile() function opens the file that SHOULD exist at this time. However, if it does not for some reason, it does make a new file with this name. Then we write in the same format for every user we have, but not after encrypting the password as we write it.

The last main class is the **HighscoreManager.java**, which keeps track of scores for single-player games:

- 1) The purpose of this class is to save the highest scores of the games that have points: single-player games.
- 2) The constructor, like the *UserManager.java*, is called when the instance is made in the *GUI_window.java* file. The constructor has to be manually updated with the symbols of the games that we are to keep track of. We do so by adding to the `List<String>` symbols as so: `gameSymbols.add("MG")`. This adds this game symbol to look out for when we are reading the file we have saved. Then we add the same number of instances of game symbols to the username and highscore list. This overall just keeps the game symbol, username, and high score at the same index in each `List<String>`. This section here could have been upgraded by making a class that will hold each of these. Then lastly, the constructor will call `loadFromFile()`.
- 3) The `loadFromFile()` function will open the file or make a file with this name if it does not exist yet. Then it will read the file, looking out for those symbols. When it finds one, it knows that the next two lines will contain the username and the high score for that game. It will find the index where this symbol is and use that index to save the corresponding information. Once everything is done, it will close the file.
- 4) There are two get functions in this class, which are `getUsername()` and `getHighscore()`. These two get the data we are looking for while also being safe for returning when there is an error.
- 5) Then there is the main function of the show: `reportScore()`. This function takes in the game symbol, the user that is currently playing, and the attempted score. The function then finds the game and checks if this new score breaks the current record; if it does not, then it just returns false. However, if it does break the current record, including if the record has not yet been set, then we save information into the list that we have in there and call the `saveToFile()` and return true to let the caller know.
- 6) The `saveToFile()` writes the list that we have into the same file that we are reading from, in the same format that we expect it to be in `loadFromFile()`.

Using these three classes, the games can now be modular, and I can add as many as I want. The games now have an environment where users can play them, they save the points that are made in the single player games, high scores are password protected, and all is not volatile memory.

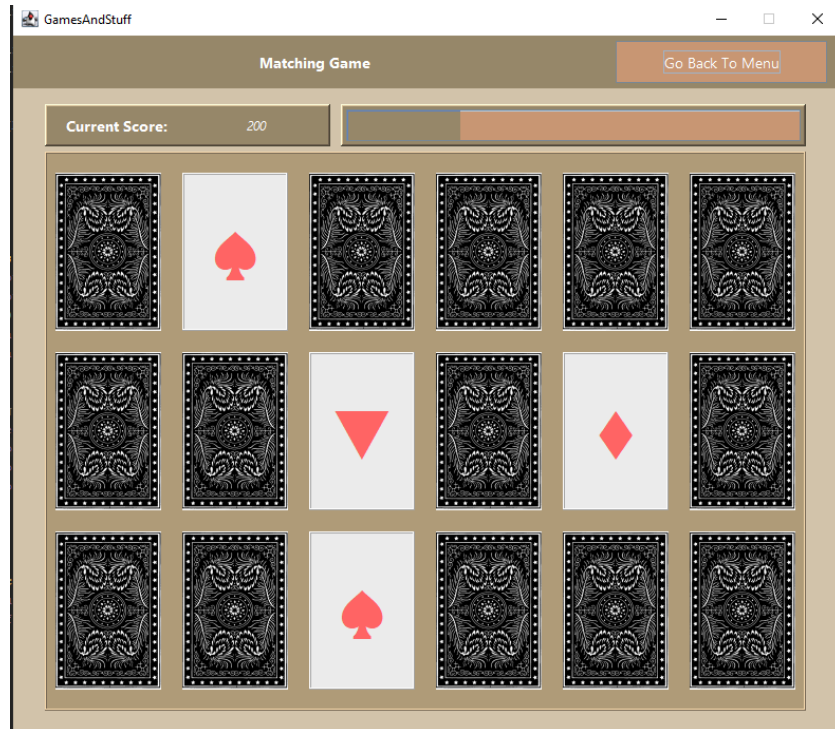
Now we can move on to show how each of the modular games works and how the user plays them as well.

MatchingGame.java

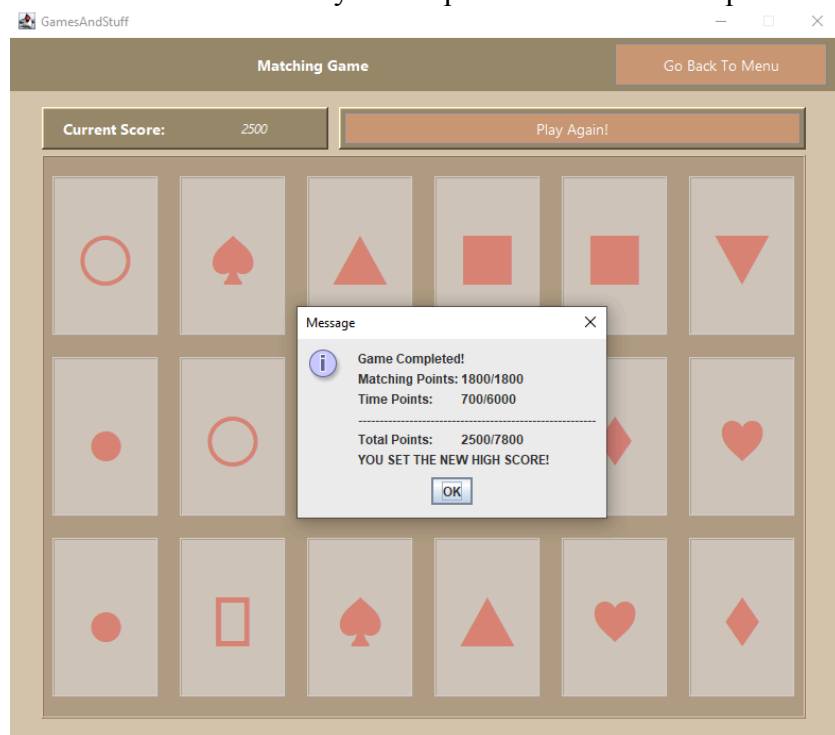
The matching game follows a regular matching game logic. All cards are laid face down, and the users flip one card and then a second. If the first card matches the second card, then the two cards stay facing upward. However, if they do not match, then the cards flip back to facing downward. The goal of the game is to make all the matches before the time runs out. The triggers for the game to be finished are that all cards are flipped upward. The points for this game are calculated using how many matches were made and how much time the user still had left when the game was completed.

The code logic flows as follows:

- 1) The game restarts, flipping all the cards facing downward. In the code, this means that we hide the label that holds the text of a single symbol and show the label with the image of the back of a playing card. Then the cover, which is a translucent panel, is set to visible so that the user is unable to click on cards. The card array is shuffled in the background, and some other visuals are also reset. Now the game is waiting for the user to click the start game button.
- 2) To shuffle the cards, we first find the correct
- 3) When the start game button is pressed, the cover is removed and the timer starts.
- 4) When the user clicks a card for the first time, the handler finds out the index of this label and sends it to the method `selectCard(int)`. If this was the first time, we set a variable called `selection` to this index, hide the image label, show the text label that corresponds to the index, and return.
- 5) When the user clicks a card for the second time, the handler finds the index again and sends it to the class again. We flip the cards the same way as before, but this time, since we already have set the `selection` in the previous flip, we continue to check if they match. If the two cards match, we update the score, reset the `selection` variable, and check if we have made all matches. If we have made all matches, then we can return `true` back into the original handler function. On the other side, if the cards do not match, then we set a boolean called `busy` to `true` to not allow the user to click another card, then we set up a timer that, after a (`PREVIEW_TIME`) seconds, flips the cards back over, resets `selection`, and sets the `busy` back to `false`.
- 6) In the click handler, after each click, we check if the class method returns `true`, indicating that the game has finished.
- 7) If the game has finished, then we call `stopGame()`, which ends timers that were set inside the class, adds the extra points to the score for any time that the user has left, and shows the original cover again.
- 8) We report the score that this user made to the *highscoreManager.java* and check if it set a new high score. We show this, if that's the case, along with the explanation of the score in a `JOptionPane`, and wait for the user to click restart game if they wish to do so.
- 9) The restart game button just calls `switchFrame(game1Frame)`, which is the same action that the start game 1 button does, effectively resetting the entire game from the top.



The user has made a match of spades, and has flipped two other cards that were not a match. This means that they will flip back soon after this picture was taken.



The user has made all the matches and got the end of game message. In this case, they also set the highscore and got the message of that case.

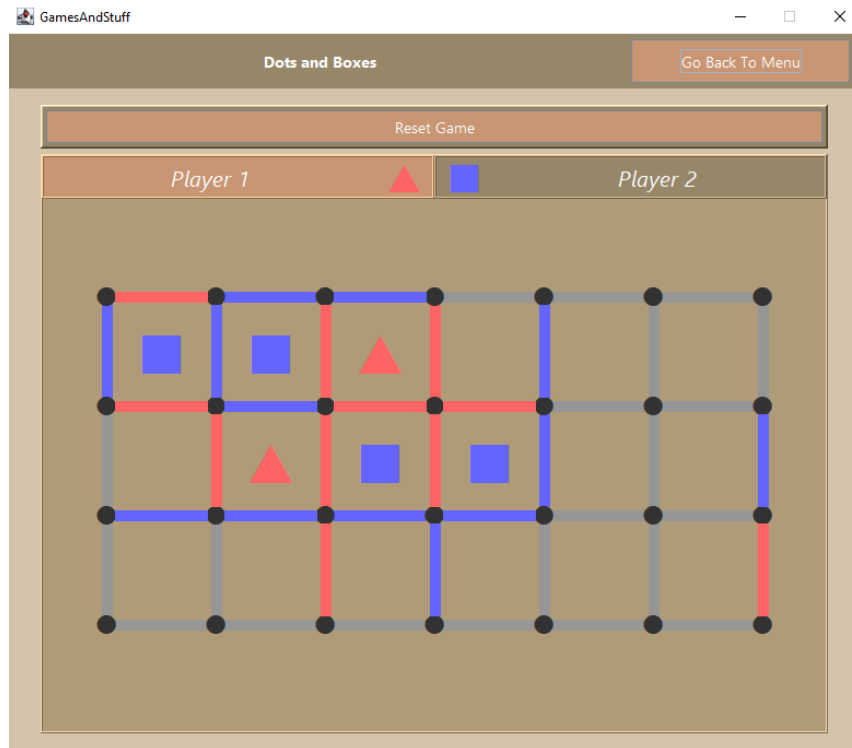
DotsAndBoxes.java

Dots and Boxes is a game where users take turns choosing a line to select and when that chosen line causes a square to be completed, the user claims that box and gets another turn to choose. The goal of the game is to capture the most boxes by the end of the game which is defined when all boxes are claimed by a player.

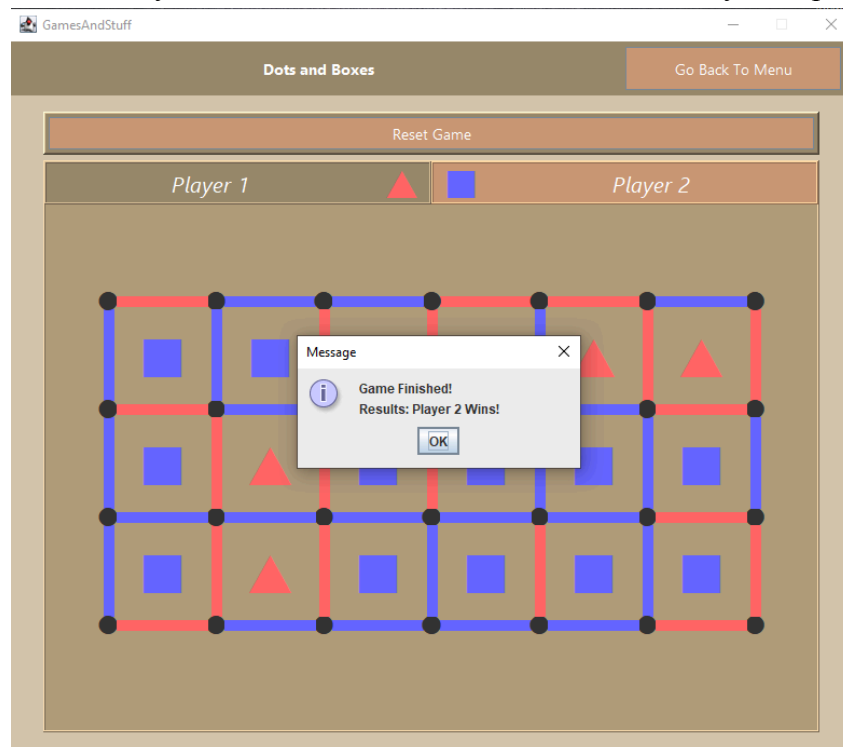
- 1) The game starts with player 1 being in turn to choose a line. All lines at this time are greyed out and when the user chooses a line, the handler of `lineClicked(MouseEvent)` is called.
- 2) The event handler checks if the game is over first. If it is not, then the panel is sent into a function inside the DB class called `lineClicked(JPanel)`. The return of this function is either true or false, indicating if the game has been finished. If it did finish, then show the message of who won by collecting that information from the DB class.
- 3) When the `lineClicked(JPanel)` function in the DB class gets called, it checks if the line has already been claimed by a user to prevent problems. Then it calls another function called `claimLine(JPanel)` that moves the panel from a list of unclaimed lines to a list of claimed lines and also checks if this move completed a box. It returns a true or false if the box was claimed with this action. We then check if all of the boxes were claimed, marking the end of the game. If there was a box claim and the game has not ended, then we do NOT switch the players; this is because the player now gains another turn. At the end of the function, we return that the game has not ended.

The logic of the game seems really easy to understand, however it was difficult to come up with a light and easy way to keep track of what defines a box. The way that I approached this problem is by getting all of the lines from the main GUI file and organizing them into arrays to be defined as a box. This way I know what line belongs to what box. This was the main hassle with the program because there was a lot of setting and typing, but it is now an array and is quite light on the memory. This way, we know that a box is claimed when all of the JPanel lines in a box have been moved to claim after the selection was made. When this happens, we move the box to claim as well to no longer keep searching through the full list of boxes. The box goes to whoever the turn was at the moment of claim.

In this file, we also have the `resetBoard()` function that calls two recursive functions that were required by the assignment. It calls `resetLinesRecursively(int)` and `resetBoxesRecursively(int)`. The `resetlines` simply go through the boxes and clear the visual colors from the lines and reset the cursor indicator back to one that shows the as clickable. On the other hand the `resetBoxes` function goes through the boxes and sets them back to invisible to show that they are not claimed by a player.



Here it is player 1's turn and they can click any of the grey lines that are still not claimed. There have also already been some boxes that have been claimed by both players.



This is the end of game message that shows who has the most boxes and has won the game.

PingPong.java

Ping Pong started as a single player game that was going to be the basic game of ping pong against a “computer”, but eventually after some of my testers showed interest in a two player version, I also added that feature. The game is the generic game of ping pong where two players (or one player against a “computer”) bounce around a ball and protect it from reaching their goal. When the ball makes it in, it counts towards the points. When in single player, the score is kept differently so that the high score can be reported at the end. When in two player mode, the score is not reported as it is a game for just fun at the moment instead of a competitive one against a scoreboard. Below is the flow of the game and how it works:

- 1) In the main GUI form, the user can choose between two player or single player modes. The single player mode will allow for input from the user to only be from the arrow keys and the player on the right side is controlled by a “computer” instead of another player. This “computer” is really easy at first, but then becomes harder as it goes by increasing its “step”. The “step” is how many pixels the player can move per game tick, the more pixels it can move compared to the actual player means that it would be technically faster.
- 2) In two player mode, the users can input either from the arrow keys or from the “W” and “S” key to control the corresponding player in the game. There is also a secret key of “P” to pause the game mid-way that some of the testers requested.
- 3) In the PP class, we have a lot of functions doing their own things to help the game feel smooth, but the most important one of them all is going to be actually a variable called “clock” which controls the entire flow of the game. It gets started when the user clicks the start button, whether it be in single or two player mode. Explaining the clock will explain the actual flow of the game better as it is a Timer type that at each tick does everything inside the clock. Since this was my first attempt at constantly moving objects with directions, I took a really crude way at the movement decisions by saving the possible moves using arrays. However, as we will see in later games I moved towards a physics approach since we already had access to time.

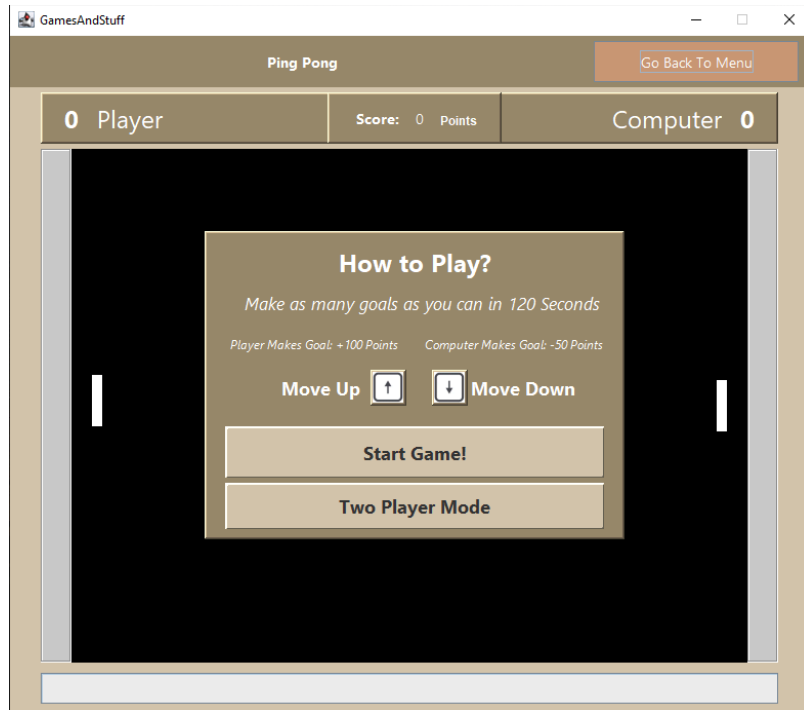
Below is the clock flow that is done at every (GAME_TICK) seconds:

- If we are in two player mode, we want to check if we need to boost the ball. Every 5 seconds, the ball gets boosted by increasing the steps that it takes. If it is right before its time to boost the ball, we start blinking the ball red to show this. Then we boost it by upping the multiplier that is used to calculate the ball’s step later.
- We check if player 1 is pressing their up arrow, if they are then we want to move the player towards the top by (PLAYER_STEP) pixels. However, we first want to catch when this would put the user outside of the visible play area. If that is the case, then we clamp it to the top and don't let the player move. If the player is not pushing their up arrow, then we check if they are pushing the down arrow and do the same thing.
- Then we move on to the second player. In this we have the two conditions of it: the game is in single or two player mode. If we are in two player mode then we move the user in

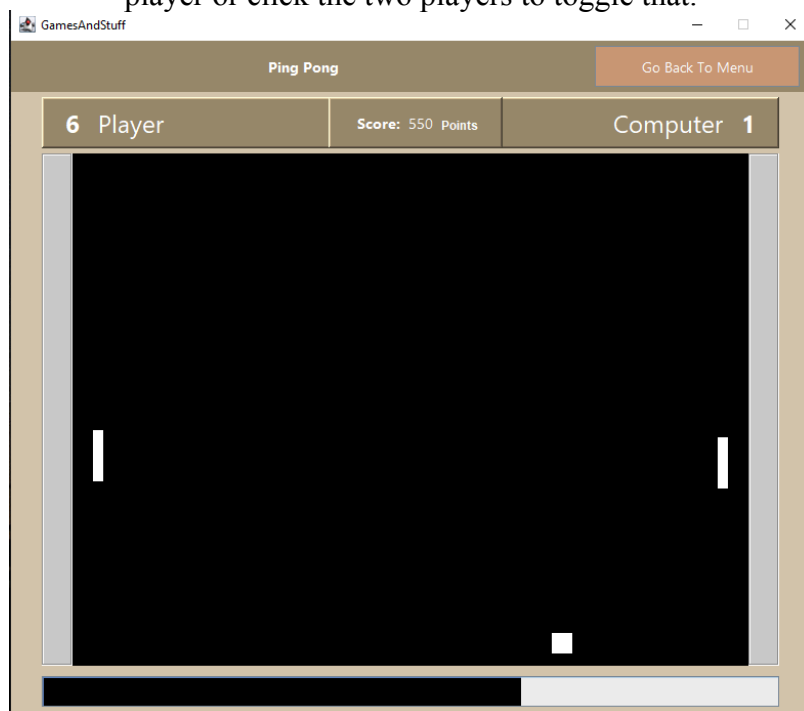
the same way that we moved the first player. On the other hand, if we are in single player mode then we want to approach it differently. If this user is a “computer”, then we check if the “computer” is going to move this Timer tick. We start the game off slowly by only letting the “computer” move every 5 ticks, this way the “computer” reacts really slowly. Then as the single player makes goals, we start decreasing this number until eventually the computer is making a move every tick just as the player is able to. However, to make it harder, once we reach 0, we then start increasing the amount of pixels that the computer can move by, while the single player stays at their same speed. This forces the game to get infinitely harder and makes it impossible at a certain point to get more points. The “computer” decides its move by simply looking at the position of the ball, if it is above the “computers” location and so forth.

- After this, we move the ball a certain direction by its step; if its single player it does not get any faster though. We first get the delta that the ball is currently moving at, this finds out what the next move should be of the ball by using the arrays. (Again, this was a really crude way of doing this, but it was my first time with constant moving objects) Once we get by how much the ball is supposed to move in the y and x direction, we move the ball to that location. However, before moving the ball to this location, we first have to check if the ball has hit a boundary, if that is the case, we want to clamp the location of the ball to that boundary to prevent errors.
- Then, we have to handle the case that the ball touches a wall. If the ball hits either the top or the bottom of the game box, then we want to change the current delta (direction) that the ball is going to travel from here. We do this by adding or subtracting 90 to the angle depending on the direction that it is going and what wall it hit.
- After handling the top or bottom, we have to handle if the ball hits the left or right. This section was long because the conditions the ball can be in is that it either hits the left goal, the left user, the right goal, or the right user. Along with these, we want to also prevent the ball from being caught in a loop. The way we do this is by checking if either user was currently moving in either direction, doing so also affects the angle. This way the user can catch another user off guard from expecting a 90 degree bounce.
- When the ball was moving towards the left side, we have to check the condition of if the ball hits the person or it got past the person and hit the goal that they were defending. If it hits the person, then we check if the extra angle was gained and then bounce the ball similar to the way we bounce it off the top or bottom of the game box. The right side is handled the exact same way with different mirror numbers since it's going in the opposite direction.
- The very last step is to check if the ball hit a goal, if it did then we report this to another function that handles who to give the score to and also takes into consideration if we are in single or two player mode to handle scoring differently.

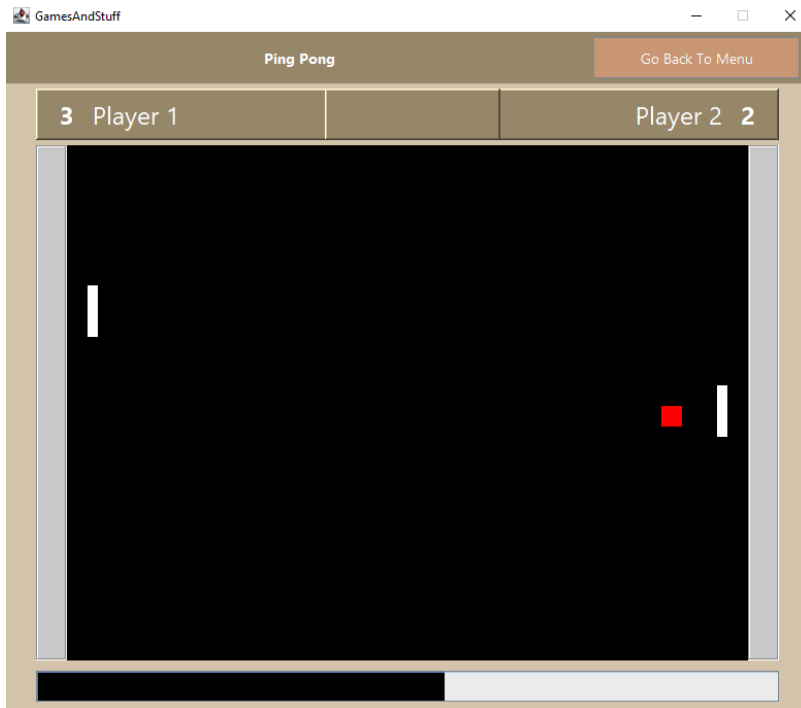
The rest of the code are helper functions that take care of smaller operations or visuals such as the ball boosting and showing scores.



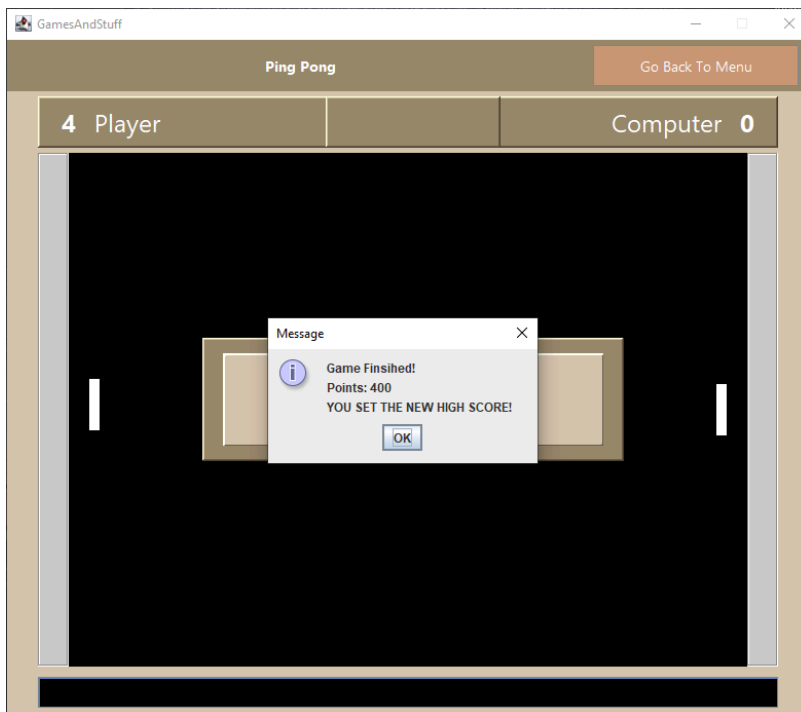
This is the starting window that the user is greeting with. They can start the game as a single player or click the two players to toggle that.



This is the single player mode where the score is shown in the top middle and the player has made 6 goals now. This means that the “computer” is now going to start stepping further and is making a move at every tick.



This is the two player mode where the score is not kept. Here one player has made 3 points and the other has made 2. This picture also shows that the ball is currently flashing to show that it is about to speed up its stepping.



This is the end of game message that shows that the game has finished in the single player mode and shows their score after it was reported to the highscore manager to save it.

TanksGame.java

Tanks is a game where players take turns controlling their tank and turret to launch a ball to their opponent in hopes of taking a heart away when it lands on them. The shot is made by holding the space button until the user wants to release it, giving the ball a stronger initial velocity. This continues until one player loses all their hearts. The game follows a format of playing 3 matches until one player wins 2 out of the 3 maps. This will declare that player the winner of the game. Similar to the Dots and Boxes game, it does not report scores. The code follows through as below:

- 1) The users are met with a screen that explains the rules and lets the players choose their first map to play at.
- 2) When the player completes that game map, they are sent back to the same page with that map covered with a translucent marker showing which player won that map. Then they can choose a second map to play at.
- 3) This continues until 2 out of 3 maps are won by either player. Then the winner is declared and it shows a form of a reset button in case they want to try again in a different pattern.

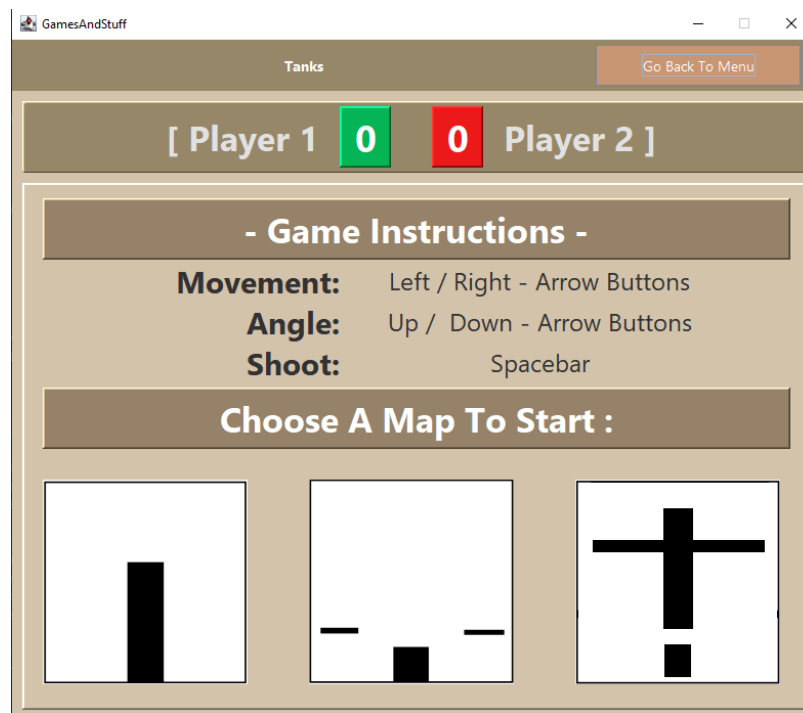
The game class, like the Ping Pong game, is also relatively long. Therefore, the game clock will be explained as well:

- First we have to check if the player is falling. This can only happen in one of the maps where the tanks start at a higher location and they can fall down if they step off the ledge. If their positions decide that they are falling, then we stay in an inner loop of the Timer where the player's Y location is increased every tick by (FALLING_STEP). When they reach the floor, then the loop breaks out and we can continue onto the next parts.
- Then we can check if the ball is not flying. If the ball is not flying, then we can check if there is a move that the user has pressed. When they have pushed something, we call movePlayer(String) that moves the player or moves the turret, depending on what move is called for.
- However, if the ball is flying, then we stay in the loop of the ball flying. If this is the case, then we then have to check the two conditions of if the ball has hit a player or if it has hit a wall. If this is the case, then we can check if the ball hit specifically a player. If it does then we can show the explosion.
- When we entered into this larger If statement we increased the cyclewaited variable and after enough Timer ticks passed, the (cyclesWaited >= CYCLES_WAIT_AFTER_IMPACT) became true. This lets the function enter into that section where it hides the explosion again and takes a heart away from the player that was hit. We take away the heart in a takeHeart() function which returns true if the player has lost all of its hearts. If this is the case then we stop the timer, save what player won, change the score, and send the user back to the starting window.
- After all of the ball hitting handling, we can then set the ballIsFlying back to false, change the player, and reset the gas and power bar.

- While that was all inside one large if statement of if the ball has hit something, in the condition that the ball has hit something, if the ball has not yet hit something, then we simply call the function of moveBall().

Inside the moveBall() function, it is MUCH shorter for calculating where to put the ball next because we are using the advantage of the Timer. We can simply calculate the next position by using a velocity on both the X and Y axis. This way we can also use a negative velocity to represent gravity. This all sets the new location of the ball.

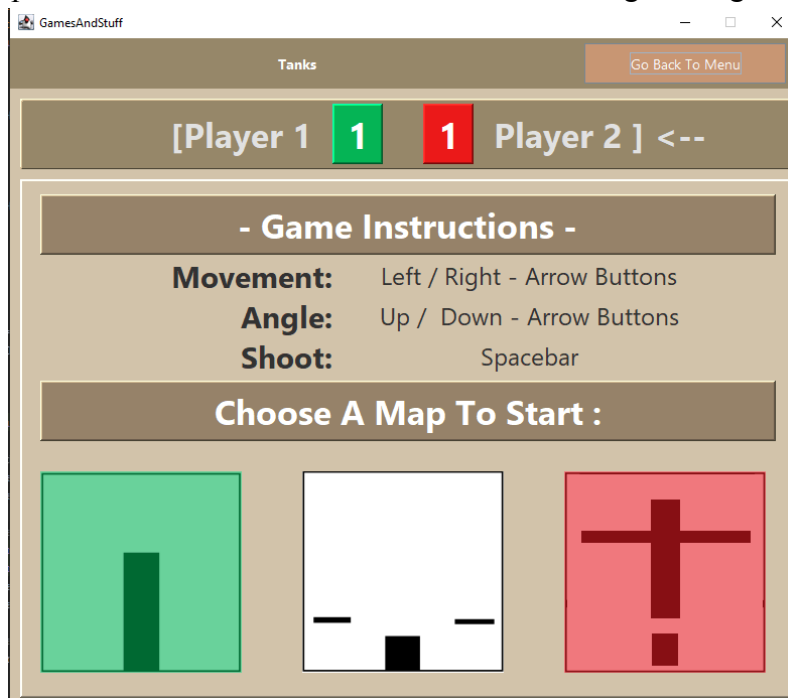
Aside from that major Timer, there is another section that moves the turret that was hard to code. To show the turret, I had to create a class that extends the JPanel so that I can control how it gets painted. This allowed me to send into the class a certain angle and the panel rotates by that much, as a turret would. One of these objects is given to each of the tank's JLabels to show this turret moving as the user pushes the up and down arrow buttons.



This is the menu that the users are greeted with to choose their first map to start the game.



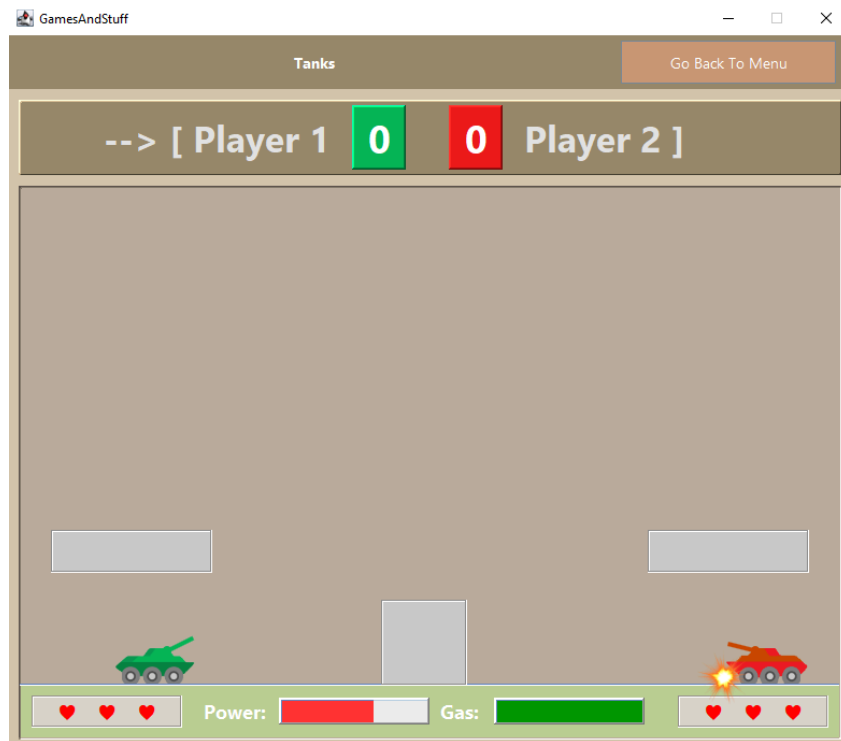
This is the map that allows the user to fall and also shows the gas being used by player 1.



This shows what it looks like when each player wins one game and we are waiting for the next game map to be chosen.



This picture shows the red tank shooting their ball across the map after holding the spacebar for almost the entire power bar amount. The ball's trajectory shows that it's going to hit the wall before it hits a player so this would be considered a miss and the turn will be passed.



In this frame, it shows the green tank took a shot that landed on the red tank, and the explosion was shown. After the time of showing, the heart will be taken away from the player.

Tag.java

Tag is a game where each player controls one user and tries to tag their opponent when assigned the role of the Tagger. After the user tags their opponent, they are frozen and are assigned to be the new Tagger. The game continues until one player reaches the Max Tagger Time. The player who reaches that time loses. The game also has a boost that can be collected, making the user who collects it run faster. In this specific game, the players can manipulate some of the game variables: Max Tagger Time, Runner Speed, Tagger Speed, Time Frozen, Boost Speed, Boost Respawn Time, Boosted Time, and Gravity.

The game starts at the menu that explains the game rules and allows the players to enter the settings section to change any variables they wish to. When the players are ready and push the “Press to Start” button, they are sent into the game box where the starting Tagger is shown after they have been chosen randomly. The users can then click the “Start Game” button to actually begin the game and the timers.

The flow of the game follows a game Timer similar to the other games. The logic for that time is as below:

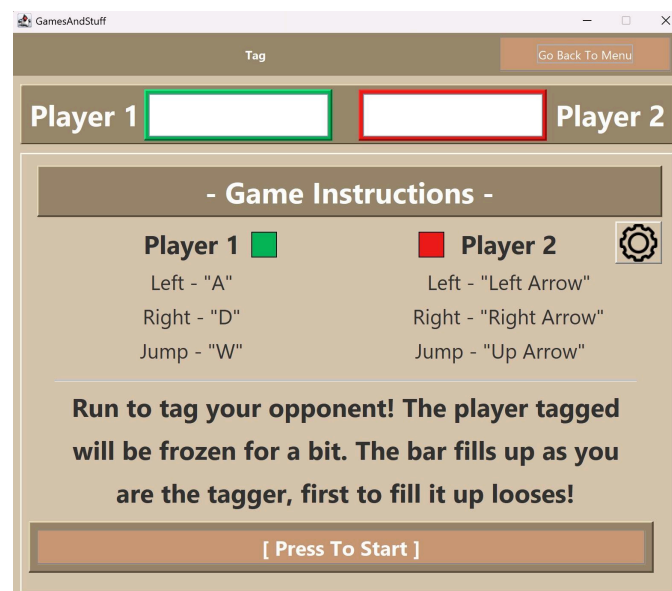
- The first step is to add a second to the Tagger’s progress bar. We do a simple if statement to find out who the tagger is, since the physical progress bars are different. After increasing the time, we check if this made the Tagger’s progress bar reach its maximum, meaning the end of the game. If it did, then we would stop the game timer and show the results through a JPanel and a JLabel.
- Then we handle player 1’s X movement. Before we allow the player to move this tick, we check if the player is frozen. This would be the case if this player has just gotten tagged and is frozen to allow the other player to start running away. I
- We handle the left movement by checking if the user CAN move and is currently pressing their button to move left. If these two are true, then we can also make sure that moving the player to this position will not put them out of bounds of the game box. After clamping that leftward position, we can move the player their number of steps to the left. Their steps depend on whether they are a TAGGER or a RUNNER and the p1Boost. The p1Boost is set to 0 if the user does not have a boost activated and set to BOOST_STEP if they do.
- The right movement is checked in the same exact way as the left movement while also clamping to the right, making sure that the player does not get pushed outside of the game box.
- Then the condition of player 1 is frozen is handled. The game tick function would not enter here if the player is not set to frozen. However, if they are frozen, then we increase the tick counter of the times we have been at this position. Once we have reached the TIME_FROZEN amount of ticks, then we can unfreeze player 1 and reset the counter back to zero for the next time that they are frozen.

- The last thing we have to check for player 1 is if this move that we just made in the X direction caused the player to “fall off” an edge. This is handled outside through a function called `isPlayerTouchingFloor(JPanel)`. To summarize what the function does, it goes through a saved list of panels that we have defined to be the visual floors and checks if the positions that the player that was sent in through the argument of type `JPanel` is currently on top of one of these panels. If we go through all of the floor panels and see that the player is no longer touching one of these, then we can go ahead and return false (that the player is NOT touching the floor). Back in the Timer, if this is the condition and the player is not frozen (preventing what I called “dead drops”, where the user could not move left and right but only falls down), then we can set the variable `p1OnFloor` to false, and set player 1's Y velocity to the saved gravity velocity. (We prevent exponential gravity by checking if their velocity at this very tick is currently set at 0)
- Then we follow the exact same logic for player 2. From handling the X movement of the player all the way to checking if the player is no longer touching a floor panel. The only difference here is going to be the `JPanel` that represents the player and other variables that are unique to that player, such as the `p2Boost`.
- After handling the X-movements of both players, we then have to handle their Y-movements. Starting with player 1, we check the variable that we set earlier called `p1OnFloor`, along with the `p1Frozen` together, to enter an if statement that applies the y movement for this tick frame. At every tick, we have to apply the gravity acceleration just as we do in free-fall diagrams in physics. After doing so, we can also set a terminal falling velocity that we do not want the players to pass to prevent an extremely fast fall. That is done by just clamping the speed at which we are currently falling.
- Then we move player1 according to the `p1VelocityY`. When we are falling, the only thing that affects this is the initial (gravity) velocity that we set up. However, if we jumped, then the initial velocity would actually be set in the opposite direction of gravity. This was in the Y-axis, we can form an up and down motion as the user spends tick time.
- We can now check if we have hit the top of a panel. We do this by checking the y positions of every floor panel that we have against the position of player 1. If this is the case, then we reset the velocity to 0 so that at the next tick, we can apply only the gravity again. This visually shows a form of hitting the top and being stopped, preventing the user from going through the floors upward.
- Lastly, we check to see if this Y-movement now puts the player on the floor or any of the panels that we have also defined to be visual floors. If so, then we can “snap” the player onto the top of the panel so that the `JPanel` does not go through the floor downward.
- Then the same exact same logic is applied to player 2, from the moving player in the Y-axis to “snapping” the player to the floor.
- After both of the players' up, left, and right buttons were handled, we can now check the movements caused by this tick that made the players touch. We do this outside in another

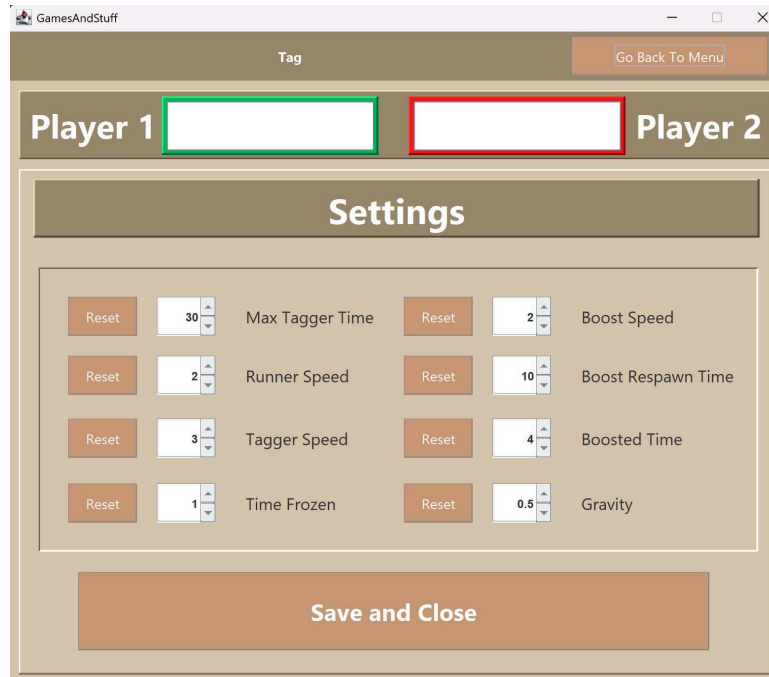
function called `handleTagCollision()`. To summarize the called function, we check using Rectangles if the players have intersected each other. Then, if true, we change the roles of the two players, freeze the new Tagger, and push the two players away from each other to give the Runner some more space. Along with this, we can also change the visuals of the game, such as the indicator that shows which player is currently the Tagger.

- From here, the only thing left to check for would be the boost conditions. We first have to check if the boost is available to pick up, after which we then check if there was a collision between a player and a boost. If there were, then we could go ahead and change the corresponding boost. However, if the boost is not available to pick up, then we can continue a countdown for the boost to be “respawned” at its location. In this else statement, when the counter finishes, we can then move it back to its position, and there is the possibility of going into the original if statement.
- The same thing happens for the second boost at a different location. All of the logic is the same, but with different objects, so this must be typed out differently.
- Lastly, we have to check if at this tick, the player has run out of time to use the boost. We do this by checking if the `p1boost` and `p2boost` are currently set to the `BOOST_STEP`, which is the number of extra steps it takes when they are in a boosted mode. If the player is boosted, then we can continue counting towards the catch of reaching the `BOOSTED_TIME`. This is when we reset the `p1boost` or `p2boost` back to 0 and reset the counter.

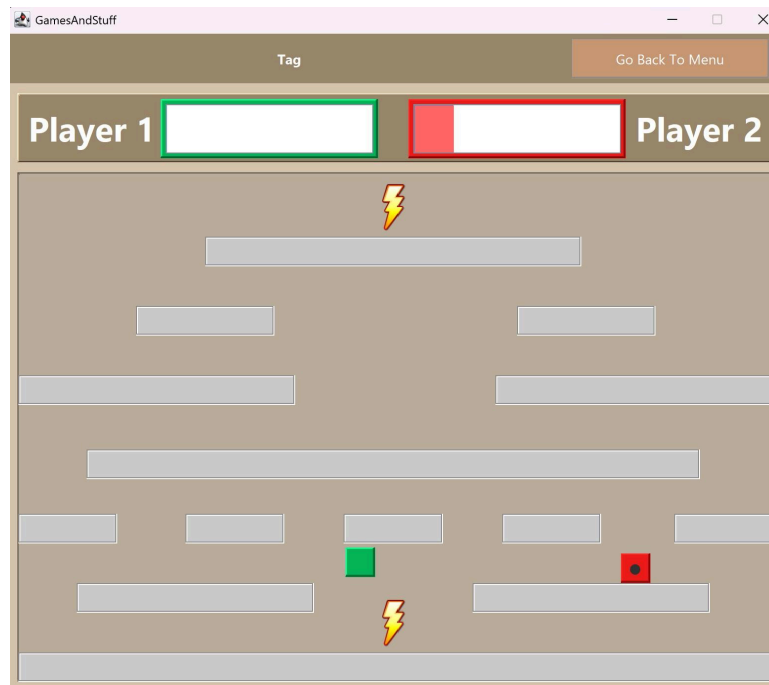
In this program, I learned a much easier way to handle the movement of loose objects using gravity and the idea of acceleration, along with grouping the panels for a shorter code of looping instead of going through individual panels. I was able to see my growth in efficiency through this game, but I knew there was still much to learn before I could tackle my next game.



This is the starting screen that the user sees when they start the game.



This is the settings tab that allows the user to customize their game match to make it more fun.



This is a game in session. The red user currently has the indicator that they are the tagger. The green user has jumped and is going to grab the boost that is right below him. This will make his speed greater than the Tagger's now. The two bars on top show the amount of time as Taggers.

CastleDefense.java

Castle Defense is a game where enemies follow a line towards a tower and the player places towers on the field to attack these enemies before they reach the castle. Towers can be bought with money that is given at the start and at checkpoints and when enemies are destroyed. When enough enemies reach the castle and bring its health down to 0, the game ends and the score is reported to the highscore manager for the user that is signed in.

The Castle Defense game was the biggest game that I made for this project. I spent time improving my skills in Java and Netbeans to be able to give this game my larger attention. This file includes the Tower class, Enemy class, Projectile class, and the CastleDefense class itself. To understand a lot of what is being processed in the game timer for this game, we have to go over each one of these classes briefly.

Tower Class:

- This object is made when a user buys a tower and places it in one of the locations that is available. The tower constructor needs information from outside such as the tower type, the placement it was put on, the upgrade costs, the power list, range list, ability list, its reload time, its projectile color, and a couple more.
- One of the main methods for this class is the upgradeCategory (int, int). This method checks if the category that is sent in costs less or equal to the cash the user has that is also sent in. Since every category has its own pricing at different levels and the user cannot upgrade past the third level, this is all handled in here and returns at the end true or false corresponding to if the upgrade purchase was successful.
- A second important method is the update(ArrayList<Enemy>). This method is called in every game tick and its purpose is to handle the move of this current tower for this current tick. It uses private helper functions to find a target to aim at and to check if the enemy is in range. The method also handles the counter for if the tower is currently reloading to know if it is even able to shoot. Lastly, the method returns true or false corresponding to if the tower is ready to shoot or if it has no target or the reload counter is not ready.

Enemy Class:

- This object is made a certain amount of times corresponding to what round the user is currently in when the user clicks the start next round button. Its constructor needs information from outside such as a JLabel that will show the image, the health that this enemy starts with, the damage this enemy starts with, and the line the enemy is currently on.
- The only important functions in this class would be the setLocation(int,int) that moves the JPanel that we saved in this object and the takeDamage(int) that hurts the enemy's health along with returns if this damage has killed the enemy.

- Aside from these two, the rest of the methods are get and set functions. This is because this class is mainly acting as a holding class and one to be targeted by other objects such as the towers and projectiles.

Projectile Class:

- An instance of this object is created every time a tower is ready to shoot and the count and type of projectile depends on the type of tower that is producing the projectile. The constructor needs from the outside such as the Tower that shot the projectile, the type of tower so that we can control the projectile, the damage that it does, the projectile color, and much more.
- In the constructor, we also handle a case that if the projectile is a tower 3 type, then we have to draw a missile. This was my first time experimenting with drawing custom panels, but I was able to make it work. On the other hand, if the projectile type needs to be a bullet from tower 1 then we just set the sprite to be a square with specific dimensions. For a tower 2 type, we do not draw anything specific because we actually force it to later be a lightning.
- The first important method is the `update(ArrayList<Enemy>)` method. This method starts by checking if the targeted enemy is already dead, if they are, then we can just remove this projectile. Then we have to consider each projectile type, we do this through the `towerType` instead of detecting the `towerType` from the tower that was set because one of the towers has to make one of each projectile. Each projectile type does different targeting protocols. Type 1 tower shoots at the enemy but the bullet follows the enemy even if they move from the original location. Type 2 tower shoots an electrical shock that bounces from enemy to enemy as long as they are in reach and the tower's upgrades allow for it. Type 3 tower shoots a missile that goes to a specific location and locks onto that location instead of chasing an enemy.
- Then there are `createLightningLine(Point, Point)` and `createExplosion(Point, int)` that draw their corresponding visuals. The lightning method deals the damage instantly and shows where the lightning goes by overriding a `JPanel` and its `paintComponent(Graphics)` method. I learned this when working on another assignment for a different class. While the create explosion method draws the radius that the explosion reaches and deals damage to all the enemies inside this radius.

With these helping classes, we can now make the class of the game that is going to hold these variables in list to keep track of them for when we need to remove them and also so that they can interact with one another. Going now into the game timer as we have done with the other games, this timer is going to be a lot more. Along with the timer we also have a couple more functions that are used for visuals such as the method that highlights placements and shows radius. However those are very well commented and the game timer is the harder one to understand.

Therefore, below is the flow of the main game timer:

- First we check if the user has changed the status of the fast forward button since the last tick, we compare if the delay for the timer is set to a fast tick or the regular tick and compare it to the condition of the fast forward button. If there was a change, then we change the delay and restart the timer so that the next tick can take into consideration the tick speed.
- Then we check for the end of game condition. The game has fully ended when the last tick puts the castle at a health at or below 0. If this is the case, then we clear the game box of all projectiles, enemies, and lightning since these were created dynamically. Then we set up the end of game cover with the stats of the game and report the score back to the highscore manager. Lastly, we show the cover once everything is set to allow the user to play again or leave back to the menu.
- Next, we check for the end of match condition. This is when all of the enemies have been killed. If this is the case, then we remove all the dynamic objects like we did at the end of game, show the next round button, update visuals of bars, update scores, and check if we need to give the user a cash gift. The cash gift assignment is handled in the round start button clicked method. However, if the round is not over, then we can simply update the enemiesLeftBar to reflect how many enemies we have left to kill.
- Next comes the movements of the enemies. We call a method of moveEnemy(Enemy) that handles the movement for them for every enemy in the list. The method handles to not move enemies that are no longer alive and their track. The track that the enemies follow, we use the paths JPanels that we have sent into the game class to find out the bounds on when the enemies have to change their direction and they walk in this direction that their set ENEMY_STEP speed.
- After the enemies have taken their small step, we check if this puts the last enemy we sent in a position to start moving the enemy behind them. If this is the case, then we set its variable of isMoving to true so that it can be affected by the earlier function of moveEnemy(Enemy).
- Along with this check, we also have to double-check if there are currently no enemies on the board and we still have some enemies to go, then go ahead and send in the next enemy. This fixed a bug of if the towers killed all the enemies too quickly, the last enemy never made it to the position that triggers the next enemy to start moving.
- After this, we have to update every tower that we have currently set up. We do this using a for loop and calling each tower's update(ArrayList<Enemy>) method. The method lets us know if they are able to shoot again. If the current tower we are looking at is able to shoot, then we can go ahead and use a switch statement to decide on what type of projectile we are going to be launching.
- If the tower is a type 1 tower, then we check at what current level the ability of the tower is at and launch that many projectiles of bullets towards the enemies. We do this by

simply creating the projectile object and adding it to the array of projectiles; this way we can later update the projectile to start following a path through its update() function. However, since this means that all the projectiles will be created in the same tick, all bullets will be on top of each other. Because of this, I made a timer that runs by itself to wait a certain amount of time before creating another projectile behind it, therefore creating some space between each bullet.

- If the tower is a type 2 or 3, the creation of the projectile is the same except the tower type, so we create it the same with only changing the projectiles constructor to take in 2 or 3 accordingly.
- If the tower type is a 4, then we want to do all 3 tower projectiles. We do this by copying and pasting the code for the previous projectile types and letting this tower create that many projectiles. All again being added into the projectile array that will be used to update the projectile.
- Next we can move to updating the projectiles that we have saved, in this array we may have projectiles that were created in this tick or projectiles that were created in some previous ticks that are still set to being active. We update them by iterating through the projectiles and calling their update(ArrayList<Enemy>) methods. This for loop's main function is to catch exactly how many enemies were killed with this projectile because the tower type 2, 3, and 4 can kill many enemies at once. Once we get this number, we update the stats and move on.
- Still inside the for loop for the projectiles, we can also check if this update of this projectile caused the projectile to become inactive. This would happen when the projectile has reached its target and already dealt the damage that it needed to deal. If this is the case then we want to remove the projectile from the game box visually and also from the array.
- Lastly at the end of the timer, we want to repaint the game box in case some visuals got stuck. Since the tick is really short, we don't run into "lag" because the repaint will be called before any of it would be noticeable.

Outside of the timer, we do have a lot more individual loops that are inside of other methods that handle the visuals and help the timer work. The main pride of this program comes from making most of the objects in this code dynamic so that each instance of each object can run its own course and update itself without having to be babysitted by the main timer.

You can find some pictures of the game below, but nothing beats experiencing the game first hand. I had 3 testers give me feedback on this game and it made the game much more fun and intriguing. The game is designed in a way that it is impossible to reach a certain level, but the points come from how far you made it. This makes the game a really strong strategy game and focuses on what to invest in and what to sacrifice throughout.

Locating the project requirements:

The project's requirements were:

- 2 Classes
- 1 Loop
- 1 Recursive Method Example
- 1 Exception Handling Example
- > 150 Lines

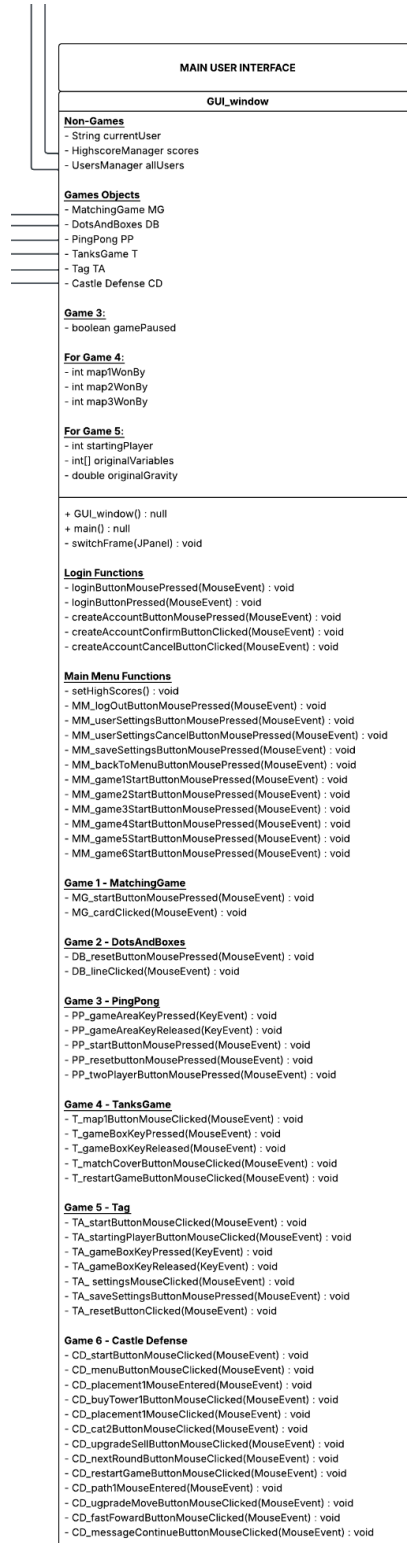
This section will simply point out the locations of the requirements since the program is long and harder to read. Please see the chart below for the locations:

Requirement	Location
2 Classes	- UserManager.java file (2 classes) - HighscoreManager.java file (1 class) - MatchingGame.java file (1 class) - DotsAndBoxes.java file (1 class) - PingPong.java file (1 class) - TanksGame.java file (2 classes) - Tag.java file (1 class) - Castle Defense (4 classes)
1 Loop	- MatchingGame.java file (line 139) - for loop in shuffle()
1 Recursive Method Example	- DotsAndBoxes.java file (lines 161 & 169) - resetLinesRecursively(int) - resetBoxesRecursively(int)
1 Exception Handling Example	- UserManager.java file (lines 150 & 183) - try-catch in loadFromFile() - try-catch in saveToFile() - HighscoreManager.java file (lines 93 & 122) - try-catch in loadFromFile() - try-catch in saveToFile()
> 150 Lines	Non-Generated Code : > 5310 lines

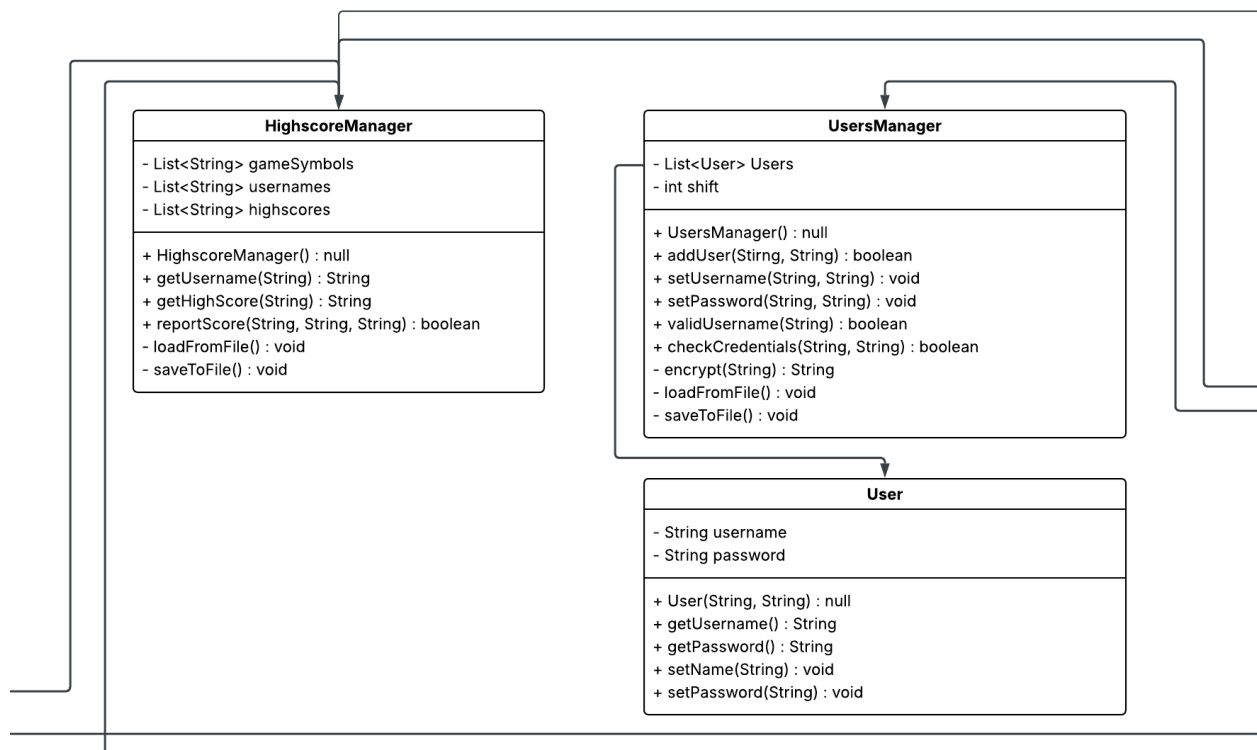
Note: I decided to use a class diagram to show the relationships between the classes and where they are part of another class to get an idea of how the entire program is designed to be modular. Link to the full diagram for easier view: [Ludid.app](#)

[illegible]

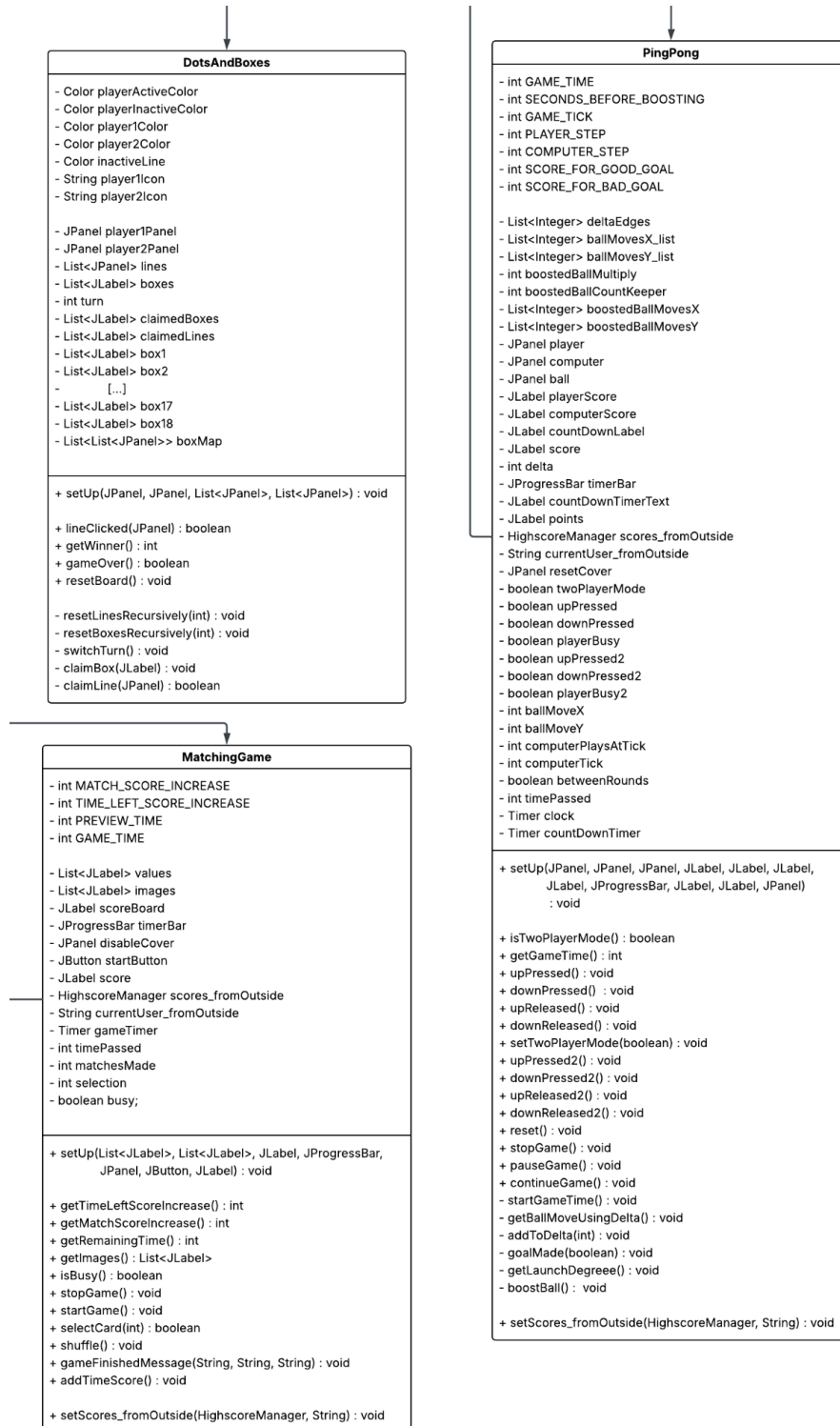
GUI Window Diagram



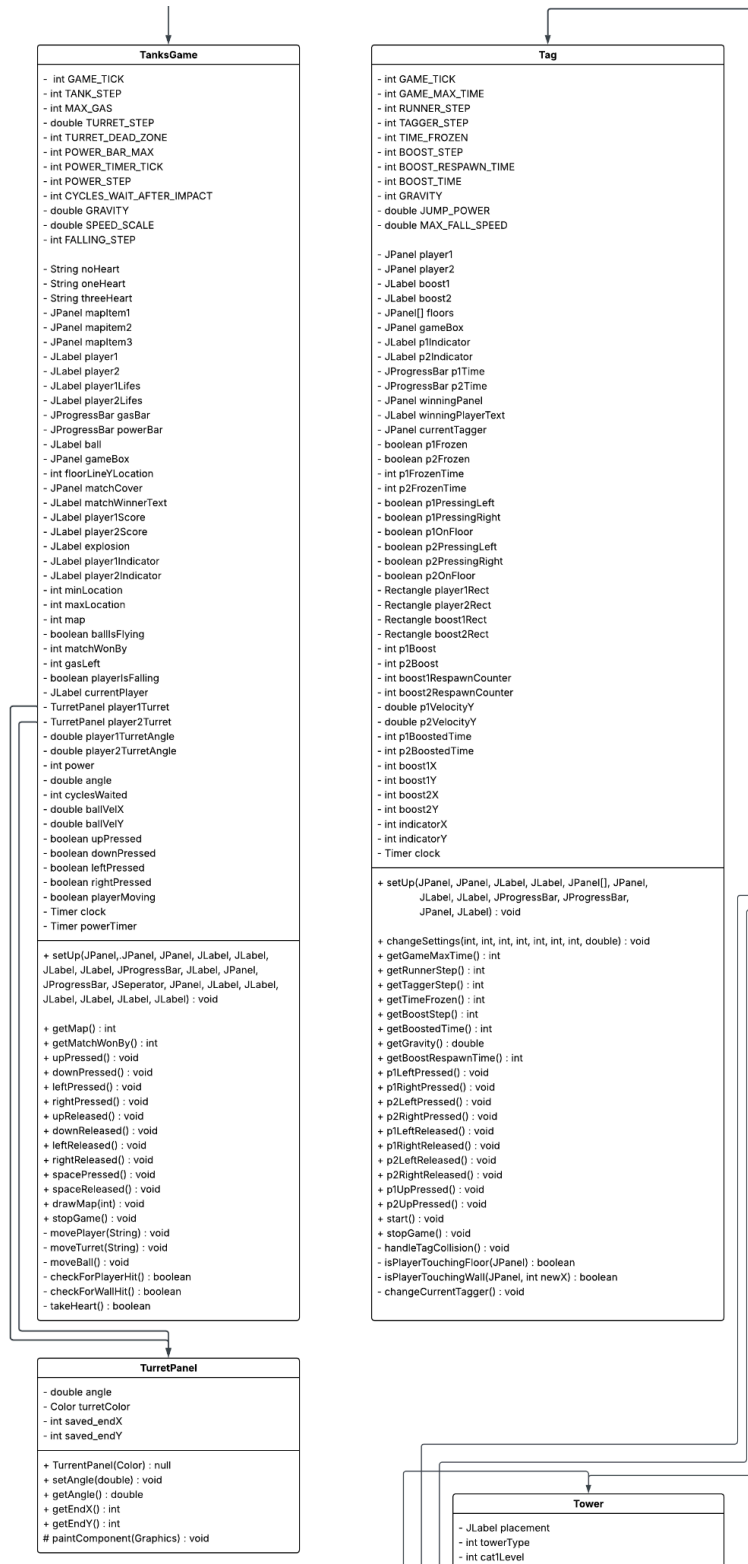
UserManger / HighscoreManager Diagrams



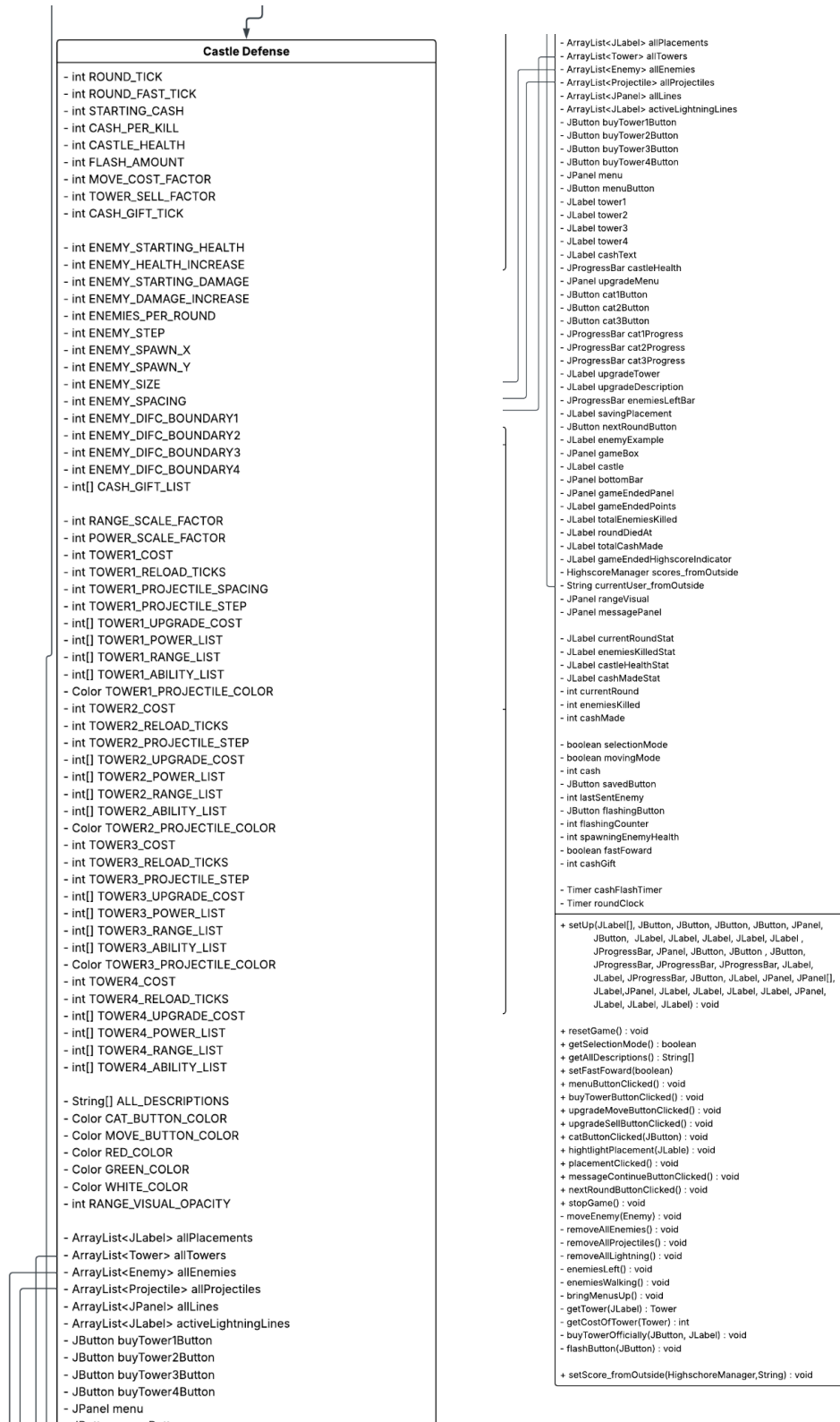
DotsAndBoxes / MatchingGame / PingPong Diagrams



Tanks / Tag Diagrams



CastleDefense Diagram



Enemy / Projectile / Tower Diagrams

